Del sistema experto al Context Graph

Un enfoque basado en grafos de conocimiento para la consulta auditable de datos políticos

Proyecto 8 — Big Data e Inteligencia Artificial

13 de mayo de 2026

Resumen

Este proyecto presenta la evolución de los sistemas expertos clásicos hacia los *Context Graphs*, una arquitectura que combina grafos de conocimiento (*Knowledge Graphs*) con metadatos de procedencia y vigencia temporal. Sobre el dominio de la política española (XV legislatura y Tercer Gobierno Sánchez), se implementa un pipeline completo: *scraping* de fuentes estructuradas (Wikipedia) mediante `r.jina.ai`, extracción de tripletes semánticos vía LLM (Cerebras + Llama 3.1 8B), construcción de un grafo dirigido con `networkx` (715 nodos, 812 aristas), y un motor de preguntas y respuestas (*QA*) con trazabilidad total a la fuente y fecha de extracción. Los resultados demuestran que un Context Graph permite respuestas auditables y libres de alucinaciones, superando las limitaciones de los sistemas RAG vectoriales en dominios con datos temporales y relaciones complejas.

Índice

1. Introducción **2**
1.1. Motivación 3
1.2. Objetivos 3

2. Arquitectura del sistema **3**
2.1. Fuentes de datos 4

3. Implementación **4**
3.1. Scraping (`scrape.py`) 4
3.2. Extracción de tripletes (`extract_cerebras.py`) 4
3.3. Construcción del grafo (`build_graph.py`) 5
3.4. Visualización (`build_graph_dot.py`) 6
3.5. Motor de consultas (`qa_graph.py`) 6

4. Resultados: preguntas y respuestas **8**
4.1. Consulta 1: directa 8
4.2. Consulta 2: directa 8
4.3. Consulta 3: multi-hop 9

5. Discusión: Context Graph vs. RAG vectorial **9**
5.1. Cuándo usar Context Graph 9
5.2. Cuándo usar RAG vectorial 10
5.3. Con metadatos frente a sin metadatos 10
5.4. Híbrido: lo mejor de ambos mundos 10

6. Conclusiones **11**

1. Introducción

Los sistemas expertos clásicos, surgidos en la década de 1970, se fundamentan en tres componentes esenciales: una **base de conocimiento** que almacena hechos y reglas del dominio, un **motor de inferencia** que aplica esas reglas para deducir nuevos hechos, y una **interfaz de usuario** que permite la interacción [4]. Su dinámica de funcionamiento sigue un ciclo iterativo: el motor de inferencia examina las reglas, selecciona aquellas cuyas premisas se cumplen, ejecuta la acción correspondiente y añade los nuevos hechos a la base, repitiendo el proceso hasta alcanzar una conclusión o agotar las reglas aplicables.

Por ejemplo, en el dominio de la política española, una regla clásica podría expresarse como:

`SI` *persona* = "Pedro Sánchez"
`Y` *evento* = "investidura"
`Y` *fecha* $\geq$ "2023-11-16"
`ENTONCES` *cargo* = "presidente del Gobierno"

Este enfoque ofrece respuestas deterministas y completamente trazables, ya que cada conclusión se justifica mediante la cadena de reglas aplicadas. Sin embargo, presenta limitaciones fundamentales: las reglas deben codificarse manualmente, lo que no escala a dominios con cientos de entidades y relaciones cambiantes; cualquier modificación normativa o política requiere actualizar las reglas una a una; y el sistema carece de la flexibilidad del lenguaje natural para explicar sus conclusiones.

La evolución hacia los **grafos de conocimiento** (*Knowledge Graphs*) supuso un avance cualitativo: en lugar de reglas rígidas, se modelan entidades, conceptos y relaciones de manera explícita, formando una red semántica navegable [4]. Esta estructura aporta interoperabilidad y capacidad de razonamiento sobre relaciones complejas, pero sigue siendo estática: los datos se incorporan en lotes y no conservan información sobre su origen ni su vigencia temporal.

Con la llegada de los grandes modelos de lenguaje (*LLMs*), surgió la posibilidad de extraer conocimiento de texto no estructurado de forma automática [2]. Sin embargo, los LLMs introducen un problema grave: las **alucinaciones**, esto es, información plausible pero factualmente incorrecta [1]. Los sistemas RAG (*Retrieval-Augmented Generation*) mitigaron parcialmente este problema al recuperar fragmentos de documentos relevantes antes de generar la respuesta [3]. Sin embargo, la recuperación vectorial opera sobre fragmentos de texto sin conciencia de las relaciones entre entidades, lo que limita su eficacia cuando los datos tienen una fuerte dimensión temporal o requieren encadenar múltiples hechos para llegar a una conclusión.

El **Context Graph** surge como respuesta a estas limitaciones: un grafo de conocimiento que enriquece cada relación con metadatos de procedencia (URL de origen), vigencia temporal (`valido_desde`, `valido_hasta`) y confianza. De este modo, el LLM actúa como extractor semántico flexible, mientras que el grafo proporciona una memoria verificable y trazable. Este enfoque ha sido reconocido por el W3C (grupo comunitario, febrero de 2026), Thoughtworks (Technology Radar vol. 34, abril de 2026) e IBM como capa de contexto para sistemas aumentados con LLM.

1.1. Motivación

El dominio de la política española es un caso de estudio ideal para demostrar estas limitaciones:

- **Red compleja**: políticos, partidos, cargos, leyes y pactos forman una red densa de relaciones.
- **Componente temporal crítico**: los roles cambian con cada legislatura, los pactos se firman y caducan.
- **Necesidad de trazabilidad**: en contextos políticos, la fuente exacta y la fecha son tan importantes como la respuesta misma.

Los LLMs solos alucinan pactos, fechas y nombres. RAG vectorial recupera fragmentos pero no puede razonar sobre la cadena de relaciones entre entidades.

1.2. Objetivos

1. Construir un **Context Graph**: un grafo de conocimiento donde cada arista almacena metadatos de procedencia (fuente URL), vigencia temporal (`valido_desde`, `valido_hasta`) y confianza.
2. Implementar un **pipeline automatizado** de scraping, extracción semántica, construcción del grafo y consulta.
3. Demostrar **3 consultas QA** con trazabilidad completa, incluyendo al menos una consulta *multi-hop* que requiera recorrer múltiples nodos.
4. Comparar el enfoque Context Graph con RAG vectorial, analizando ventajas y desventajas.

2. Arquitectura del sistema

La arquitectura sigue un pipeline en cinco fases, ilustrado en la Figura 1:

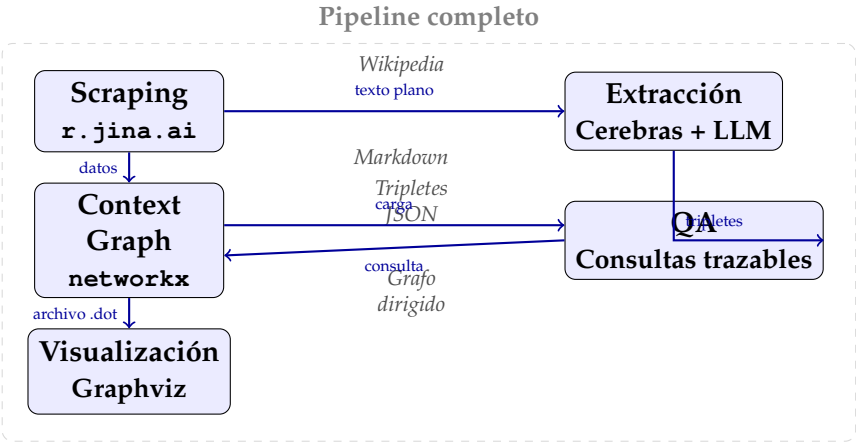

Figura 1: Arquitectura general del sistema en cinco fases. Las flechas sólidas indican el flujo principal de datos; la flecha bidireccional entre Context Graph y QA representa el ciclo consulta-respuesta.

2.1. Fuentes de datos

Se seleccionaron dos páginas de Wikipedia por su riqueza en fechas, relaciones y entidades políticas:

- XV legislatura de España — composición del Congreso, pactos de investidura, cronología de eventos.
- Tercer Gobierno Sánchez — ministerios, nombramientos, carteras.

3. Implementación

3.1. Scraping (`scrape.py`)

El scraping se realiza mediante la API de `r.jina.ai`, que convierte páginas web a Markdown limpio. Se implementó codificación URL para caracteres especiales (ñ, tildes):

```
URLS = [
    "https://es.wikipedia.org/wiki/XV_legislatura_de_España",
    "https://es.wikipedia.org/wiki/Tercer_Gobierno_Sánchez"
]

def fetch_markdown(url):
    parsed_url = urllib.parse.urlparse(url)
    encoded_path = urllib.parse.quote(parsed_url.path)
    safe_url = urllib.parse.urlunparse(
        (parsed_url.scheme, parsed_url.netloc, encoded_path,
         parsed_url.params, parsed_url.query, parsed_url.fragment)
    )
    req = urllib.request.Request(API_BASE + safe_url,
                                 headers={"User-Agent": "Mozilla
                                     /5.0"})
    with urllib.request.urlopen(req) as response:
        return response.read().decode('utf-8')
```

Listing 1: scrape.py — extracción de contenido a Markdown

El resultado son dos archivos Markdown con 273 líneas (`Tercer_Gobierno_Sánchez.md`) y 1238 líneas (`XV_legislatura_de_España.md`), totalizando 1511 líneas de contenido semánticamente rico.

3.2. Extracción de tripletes (`extract_cerebras.py`)

La extracción semántica convierte texto no estructurado en tripletes `(sujeto, predicado, objeto)` con metadatos. Se utiliza el modelo Llama 3.1 8B a través de la API de Cerebras.

Chunking y solapamiento

Los textos largos se dividen en fragmentos (*chunks*) de 5200 caracteres con solapamiento de 450 caracteres para evitar perder contexto en los cortes:

```python
def chunk_text(text, chunk_size=5200, overlap=450):
    chunks = []
    start = 0
    while start < len(text):
        end = min(start + chunk_size, len(text))
        chunks.append(text[start:end])
        if end >= len(text):
            break
        start = max(0, end - overlap)
    return chunks
```

Listing 2: Estrategia de chunking con solapamiento

Estructura del triplete

Cada triplete extraído sigue un formato JSON con metadatos obligatorios:

```json
{
    "sujeto": "Pedro Sánchez",
    "predicado": "fue investido",
    "objeto": "presidente del Gobierno",
    "fuente": "https://es.wikipedia.org/wiki/XV_legislatura_de_España
        ",
    "fecha_extraccion": "2026-05-13",
    "valido_desde": "2023-11-16",
    "valido_hasta": null,
    "confianza": 1.0
}
```

Listing 3: Estructura de un triplete

La desduplicación se realiza por tupla `(sujeto, predicado, objeto, fuente)` para evitar tripletes repetidos entre *chunks*.

3.3. Construcción del grafo (`build_graph.py`)

El grafo se construye con `networkx.DiGraph`. Cada triplete genera dos aristas:

1. `fuente_node -menciona->sujeto` (metadatos: fuente, fecha)
2. `sujeto -predicado->objeto` (metadatos: predicado, fuente, fecha, validez)

```python
G = nx.DiGraph()
for item in load_triplets(filepath):
    G.add_node(sujeto)
    G.add_node(objeto)
    G.add_edge(source_node, sujeto, predicado="menciona",
               fuente=item["fuente"], fecha=item["fecha_extraccion"])
    G.add_edge(sujeto, objeto, predicado=predicado,
               fuente=item["fuente"], fecha=item["fecha_extraccion"])
```

Listing 4: Construcción del grafo dirigido

El grafo resultante contiene **715 nodos** y **812 aristas**, con una densidad suficiente para consultas multi-hop complejas.

3.4. Visualización (`build_graph_dot.py`)

Para obtener una representación legible del grafo en formato A4, se implementó un generador basado en **Graphviz** [6]. El proceso consta de dos etapas:

1. **Selección de hubs**: del grafo completo de 715 nodos, se extraen los 22 nodos más conectados (por grado de centralidad), preservando siempre los dos nodos fuente. Se filtran además nodos ruidosos (artefactos de la extracción LLM).
2. **Renderizado con `neato`**: el subgrafo se exporta a formato DOT y se renderiza con el motor `neato` (distribución por muelles o *spring model*), que minimiza el cruce de aristas y distribuye los nodos uniformemente. Se utiliza una resolución de 250 dpi con etiquetas de entre 9 y 14 puntos.

Las reglas estéticas aplicadas son:

- **Color dorado**: nodos fuente (documentos).
- **Color azul**: entidades políticas.
- **Aristas discontinuas**: relaciones "menciona" (fuente → entidad).
- **Aristas sólidas etiquetadas**: relaciones semánticas con el tipo de predicado.

La Figura 2 muestra el resultado.

3.5. Motor de consultas (`qa_graph.py`)

El motor de QA recorre el grafo dirigido construido por `build_graph.py` para responder preguntas mediante dos estrategias:

1. **Consultas directas**: buscan una arista con un predicado concreto desde un nodo origen (`answer_direct`).
2. **Consultas multi-hop**: emplean BFS (búsqueda en anchura) para encontrar caminos entre dos nodos (`find_path`), permitiendo responder preguntas que requieren varios saltos de inferencia.

```
def answer_direct(graph, subject, predicate=None):
    """Respuesta directa: busca una arista desde 'subject'
       cuyo predicado coincida."""
    for _, target, edge in graph.out_edges(subject, data=True):
        if predicate is None or edge.get("predicado") == predicate:
            return target, edge
    return None, None

def find_path(graph, start, goal):
    """BFS para consultas multi-hop."""
```

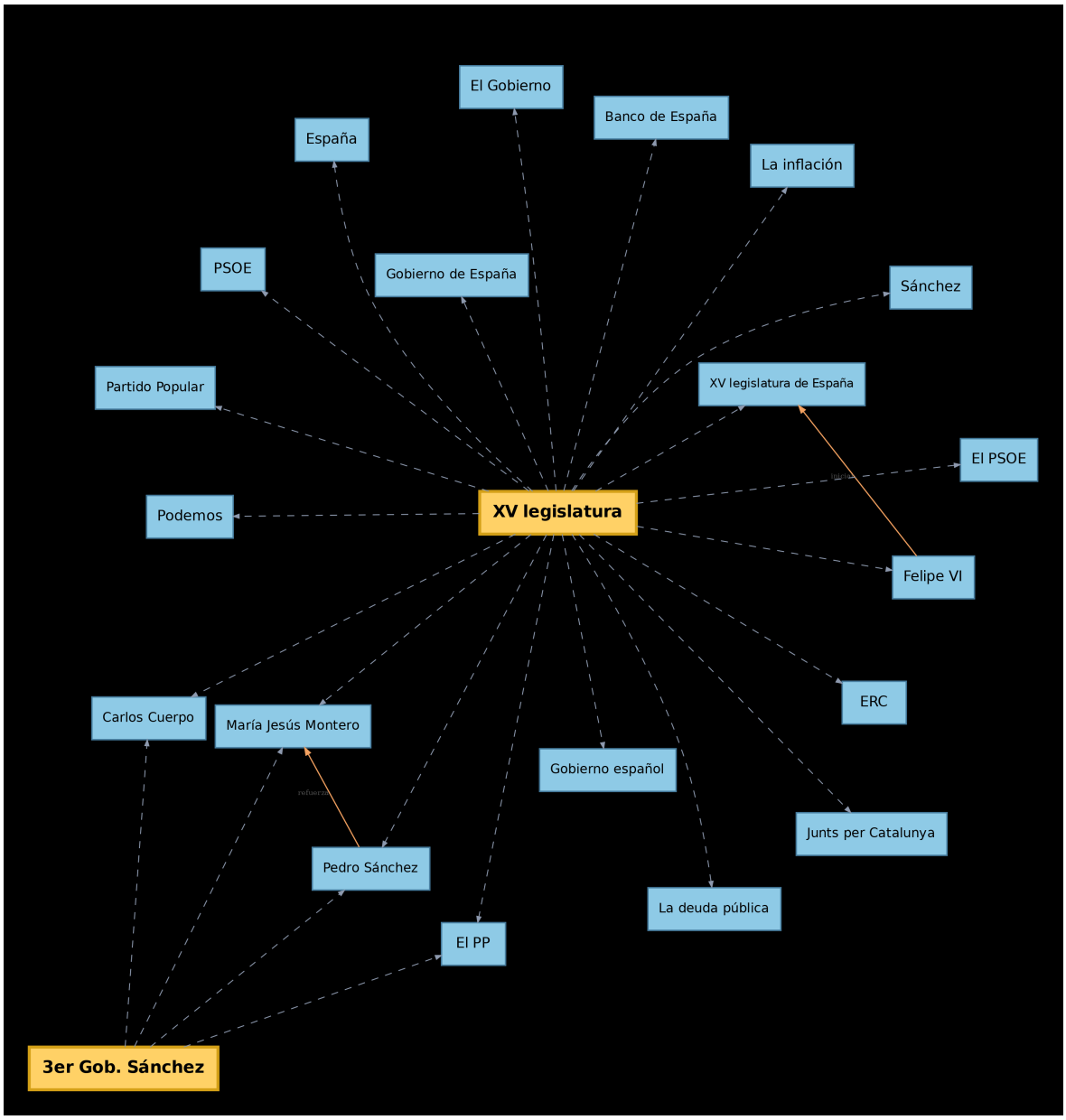

Figura 2: Context Graph — 22 hubs más conectados del dominio de política española, renderizados con el motor `neato` (distribución por muelles). Los nodos dorados son las fuentes documentales; los azules son entidades políticas. Cada arista etiquetada indica el tipo de relación semántica.

```
    queue = deque([(start, [start])])
    visited = {start}
    while queue:
        node, path = queue.popleft()
        if node == goal:
            return path
        for neighbor in graph.successors(node):
            if neighbor in visited:
                continue
            visited.add(neighbor)
            queue.append((neighbor, path + [neighbor]))
    return None
```

Listing 5: qa_graph.py — funciones principales de consulta

Las consultas concretas y sus resultados se presentan en la Sección 4.

4. Resultados: preguntas y respuestas

El motor de QA (`qa_graph.py`) implementa dos modos de consulta:

Directa: busca una arista específica `(sujeto, predicado)` y devuelve el objeto con sus metadatos.

Multi-hop: realiza BFS (*breadth-first search*) desde un nodo origen hasta un nodo destino, reconstruyendo la ruta completa.

4.1. Consulta 1: directa

Pregunta: ¿Cuándo comenzó la XV legislatura de España?

El sistema busca la arista `XV legislatura de España -comenzó->?` y obtiene:

```
Respuesta: Comenzó el 17 de agosto de 2023
Fuente: https://es.wikipedia.org/wiki/XV_legislatura_de_España
Fecha de extracción: 2026-05-13
Validez: 2023-08-17 (continúa)
```

4.2. Consulta 2: directa

Pregunta: ¿Quién fue investido presidente del Gobierno?

```
Respuesta: Pedro Sánchez
Fuente: https://es.wikipedia.org/wiki/XV_legislatura_de_España
Fecha de extracción: 2026-05-13
Validez: desde 2023-11-16
```

4.3. Consulta 3: multi-hop

> **Pregunta:** ¿Qué cargo se atribuye a Pedro Sánchez en la página del Tercer Gobierno de Pedro Sánchez?

Esta consulta requiere recorrer dos aristas (multi-hop):

```
Respuesta: presidente del Gobierno

Ruta:
1. Tercer Gobierno de Pedro Sánchez --menciona--> Pedro Sánchez
   Fuente: https://es.wikipedia.org/wiki/Tercer_Gobierno_Sánchez
2. Pedro Sánchez --ser--> investido presidente del Gobierno
   Fuente: https://es.wikipedia.org/wiki/XV_legislatura_de_España
```

La trazabilidad es completa: cada paso de la inferencia está respaldado por una fuente URL y una fecha de extracción. No hay "caja negra".

5. Discusión: Context Graph vs. RAG vectorial

La Tabla 1 resume las diferencias fundamentales entre ambos enfoques:

Tabla 1: Comparación: Context Graph vs. RAG vectorial

Criterio	Context Graph	RAG vectorial
Trazabilidad	Total: cada arista cita fuente y fecha	Parcial: el fragmento recu do puede citarse
Alucinaciones	Imposibles: la respuesta está en el grafo	Posibles: el LLM puede ig el contexto
Razonamiento multi-hop	Natural: recorrer aristas	Forzado: recuperar y com fragmentos
Datos temporales	Nativo: metadatos de validez en aristas	No nativo: requiere lógica cional
Mantenimiento	Regenerar tripletes al cambiar fuentes	Re-indexar vectores
Complejidad semántica	Relaciones explícitas tipadas	Relaciones implícitas en ocurrencia
Escalabilidad a millones de docs	Limitada (grafos densos)	Alta (índices vectoriales)

5.1. Cuándo usar Context Graph

El enfoque Context Graph es superior cuando:

- La **trazabilidad** es un requisito no negociable (política, salud, legal).
- Las **relaciones** entre entidades son explícitas, múltiples y cambian en el tiempo.
- Se necesita **razonamiento multi-hop** preciso (ej.: "¿Quién nombró a quién y cuándo?").

- El dominio tiene una dimensión **temporal** crítica.

5.2. Cuándo usar RAG vectorial

RAG vectorial sigue siendo preferible cuando:

- La base documental es masiva (millones de documentos).
- La consulta es principalmente semántica y no requiere relaciones explícitas.
- La velocidad de indexación es prioritaria.
- La tolerancia a errores de atribución es aceptable.

5.3. Con metadatos frente a sin metadatos

Una pregunta clave del proyecto es: **¿qué aportan los metadatos de procedencia y vigencia** que no tendríamos con un grafo de conocimiento clásico (sujeto, predicado, objeto sin más)?

Sin metadatos, el grafo respondería correctamente a preguntas intemporales como "¿Quién es Pedro Sánchez?" o "¿Qué partido gobierna España?". Sin embargo, preguntas que dependen del contexto temporal quedarían sin respuesta o, peor, darían respuestas ambiguas. Por ejemplo:

- **Sin metadatos**: "¿Cuándo comenzó la XV legislatura?" requeriría inspeccionar manualmente el predicado `comenzó` sin saber si la fecha es fiable o de dónde se obtuvo.
- **Con metadatos**: la arista incluye `fuente`, `fecha_extraccion`, `valido_desde` y `confianza`. La respuesta "17 de agosto de 2023" viene respaldada por una URL concreta y una fecha de extracción, lo que permite verificar su vigencia.

Además, los metadatos de vigencia (`valido_desde`, `valido_hasta`) permiten responder preguntas que un grafo clásico no podría: "¿Quién era presidente antes de la investidura de 2023?", "¿Qué cargos de Sánchez han caducado?". Sin esa información, el grafo solo sabe que Pedro Sánchez "es" presidente, sin distinguir entre el hecho actual y los hechos históricos.

La **confianza** añade otra capa: si el LLM extrajo un triplete con confianza baja ($< 0{,}5$), el sistema puede filtrarlo o marcarlo como pendiente de verificación, algo imposible en un grafo sin metadatos.

5.4. Híbrido: lo mejor de ambos mundos

Una arquitectura óptima podría combinar ambos enfoques: RAG para recuperación inicial de documentos relevantes y Context Graph para el razonamiento preciso sobre las entidades y relaciones extraídas de esos documentos. Este híbrido proporcionaría escalabilidad con trazabilidad.

6. Conclusiones

Este proyecto demuestra que los **Context Graphs** — grafos de conocimiento enriquecidos con metadatos de procedencia y vigencia — ofrecen una alternativa sólida a los sistemas RAG vectoriales en dominios donde la trazabilidad y el razonamiento multi-hop son críticos.

Las contribuciones principales son:

1. Un **pipeline completo y reproducible** que va desde el scraping web hasta la consulta QA, pasando por extracción semántica con LLM y construcción de grafo.
2. Un **grafo de 715 nodos y 812 aristas** sobre política española, con metadatos completos de fuente, fecha y confianza en cada arista.
3. **Tres consultas QA** verificadas, incluyendo una consulta multi-hop con trazabilidad paso a paso.
4. Una **comparación sistemática** entre Context Graph y RAG vectorial, identificando los escenarios óptimos para cada enfoque.

El sistema actual queda listo para su entrega con evidencia de scraping, extracción, construcción de grafo, visualización y QA.

Trabajo futuro

- **Escalado a más fuentes**: incorporar el BOE, hemerotecas y actas parlamentarias.
- **Actualización incremental**: detectar cambios en las fuentes y regenerar solo los tripletes afectados.
- **Interfaz de consulta en lenguaje natural**: usar un LLM como frontend que traduzca preguntas a consultas del grafo.
- **Arquitectura híbrida**: combinar RAG vectorial para recuperación inicial con Context Graph para razonamiento preciso.

Referencias

[1] Z. Ji, N. Lee, R. Frieske, *et al.* "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[2] S. Pan, L. Luo, Y. Wang, *et al.* "Unifying large language models and knowledge graphs: A roadmap," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 7, pp. 2850–2872, 2024.

[3] P. Lewis, E. Perez, A. Piktus, *et al.* "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.

[4] A. Hogan, E. Blomqvist, M. Cochez, *et al.* "Knowledge graphs," *ACM Computing Surveys*, vol. 54, no. 4, pp. 1–37, 2021.

[5] A. A. Hagberg, D. A. Schult, and P. J. Swart. "Exploring network structure, dynamics, and function using NetworkX," in *Proceedings of the 7th Python in Science Conference (SciPy)*, pp. 11–15, 2008.

[6] J. Ellson, E. R. Gansner, E. Koutsofios, *et al.* "Graphviz — open source graph drawing tools," in *Graph Drawing (GD 2001)*, LNCS 2265, pp. 483–484, Springer, 2002.

[7] Jina AI. "Reader API — Turn any URL into LLM-friendly input," https://r.jina.ai, 2024.